INFORME FINAL DE SCRUM — QPM CLOTHES

Sistema de gestión y seguimiento de pedidos de una tienda de ropa · SENA ADSO

Dato	Información
Integrantes y roles	Juan Quintero — Scrum Master; Marco Gonzales — Product Owner; Brayan Parra — Developer.
Jornadas / sprints	Sprint 1: 23 oct. 2026 · Sprint 2: 24 oct. 2026 · Sprint 3: 25 oct. 2026.
Repositorio	https://github.com/juanlsk29/QPM--CLOTHES
GitHub Projects	https://github.com/juanlsk29/QPM--CLOTHES.git

I. Product Goal y alcance

Entregar una aplicación de terminal clara y funcional que permita a la persona encargada de una tienda administrar el ciclo de vida de pedidos de ropa desde su registro hasta su entrega o cancelación, con datos persistentes y validaciones que reduzcan errores de seguimiento.

Alcance: registro de pedidos y prendas, listado y detalle, actualización de datos y estados, cancelación, búsqueda y filtros, historial de estados, consulta de atrasados/próximos a vencer, persistencia JSON, reportes CSV y resumen. Exclusiones: inventario/stock, cotización, interfaz gráfica o web, base de datos relacional, pagos y servicios externos.

2. Product Backlog resumido

Prioridad	Historias	Funcionalidades
P1 — esenciales	HU-001 a HU-006; HU-008; HU-010 a HU-012; HU-014	Registro y consulta, cambios, vencimientos, historial, JSON, validaciones y menú.
P2 — importantes	HU-007, HU-009, HU-013	Cancelación, búsqueda/filtros y exportación CSV.
P3 — complementaria	HU-015	Resumen de pedidos.

3. Definition of Done (DoD)

Una historia se considera Done cuando cumple criterios de aceptación, está integrada, el código es comprensible, las entradas y errores se controlan, se prueban casos válidos e inválidos relevantes, la persistencia/reportes funcionan cuando aplican, el README y la documentación reflejan la versión real y los cambios quedan revisables en GitHub. Los defectos críticos conocidos deben resolverse antes de declarar el cierre técnico.

4. Seguimiento — Sprint 1 · 23 de octubre de 2026

Sprint Goal: acordar un alcance realizable, un backlog comprensible y un plan técnico compartido antes de programar.

Planificado y resultado observado

Se planificó revisar HU-001 a HU-015, confirmar el Product Goal y las prioridades, acordar estructura, JSON/CSV, reglas de negocio, DoD, estrategia de pruebas y documentación. El historial Git conserva el commit inicial de planificación be74be7 (23/10/2026). En la copia de la rama actual, parte de los documentos iniciales ya no está presente; el historial registra eliminaciones de archivos de planificación.

ID	Historia o tarea	Responsable	Estado final	Evidencia
T1.1–T1.3	Revisar HU, alcance, prioridades y dependencias.	Marco + equipo	Registrado en planificación	Commit be74be7
T1.4	Definir reglas de negocio y criterios de aceptación.	Marco + Juan	Documentación de reglas existe en la copia; verificar publicación final.	Reglas
T1.5–T1.6	Diseñar módulos y acordar JSON/CSV y manejo de errores.	Brayan + equipo	La estructura de código está implementada.	Estructura
T1.7	Definir DoD y estrategia de pruebas.	Juan + equipo	La planificación define DoD; pruebas documentadas.	Guía de pruebas
T1.8	Actualizar repositorio y documentos iniciales.	Todos	Commit inicial visible; documentos completos no están todos en la rama adjunta.	Historial Git

Actividades y revisión

Sprint Review: la evidencia disponible permite revisar los artefactos de planificación, pero no incluye acta formal ni aceptación registrada. Retrospective — mejora propuesta: conservar documentos de planificación en la rama final y vincular historias/tareas a evidencia GitHub desde el inicio.

5. Seguimiento — Sprint 2 · 24 de octubre de 2026

Sprint Goal: obtener una versión de consola ejecutable que permita registrar, consultar y actualizar pedidos, guardar información en JSON y aplicar validaciones básicas.

ID	Historia o tarea	Responsable	Estado final	Evidencia
HU-001 / T2.3	Registrar pedidos.	Brayan + equipo	Implementado en el código; revalidación general pendiente.	consola/opciones_menu.py
HU-002 / T2.4	Registrar las prendas asociadas a un pedido.	Brayan + equipo	Implementado en el código; revalidación general pendiente.	Consola/opciones_menu.py
HU-003 / T2.5	Consultar el listado de pedidos.	Equipo	Implementado.	mostrar_pedidos.py
HU-004 / T2.5	Consultar el detalle de un pedido.	Equipo	Implementado.	mostrar_pedidos.py
HU-005 / T2.6	Actualizar los datos permitidos de un pedido.	Equipo	Implementado; requiere revalidación de las reglas de edición.	gestor_pedidos.py
HU-006 / T2.7	Cambiar el estado de un pedido.	Equipo	Implementado.	gestor_pedidos.py
HU-007 / T2.7	Cancelar pedidos.	Equipo	Implementado.	gestor_pedidos.py
HU-008 / T2.8	Consultar pedidos atrasados y próximos a vencer.	Marco + equipo	Implementado.	fechas_entrega.py
HU-009 / T2.8	Buscar y filtrar pedidos.	Equipo	Implementado.	En repositorio
HU-010 / T2.9	Consultar el historial de cambios de los pedidos.	Equipo	Implementado.	En repositorio
HU-011 / T2.9	Guardar y recuperar pedidos mediante archivos JSON.	Brayan + equipo	Implementado.	archivo_json.py
HU-012 / T2.10	Validar datos y controlar errores.	Todo el equipo	Implementado.	validar_pedido.py
HU-013 / T2.11	Generar reportes en formato CSV.	Equipo	Implementado.	En repositorio
HU-014 / T2.2	Navegar por el menú de la aplicación de consola.	Juan + Brayan	Implementado.	menu_principal.py
HU-015 / T2.11–T2.12	Consultar un resumen general de los pedidos.	Equipo	Implementado.	En repositorio

Actividades,

Completadas: código modular, menú de consola, modelos, gestión de pedidos, validaciones, persistencia JSON, reglas de negocio y reportes. Los commits visibles incluyen 942a5e0 (núcleo), be35691 (reglas/reportes) y d1e37ef (consola/documentación/pruebas), fechados 24/10.

Sprint Review: el código permite inspeccionar el flujo funcional y los archivos de persistencia/reportes. No se adjuntó acta de aceptación formal. Retrospective — mejora propuesta: probar cada historia al integrarla y normalizar valores de estado en un único lugar.

6. Seguimiento — Sprint 3 · 25 de octubre de 2026

Sprint Goal: Ejecutar pruebas de aceptación, completar documentación incluyendo readme del codigo y preparar la entrega.

ID	Historia o tarea	Responsable	Estado final	Evidencia
T3.1–T3.2	Revisar defectos y completar documentacion	Todos	Terminado	Docs/08_plan_de_pruebas.md
T3.3–T3.4	Completar y subir el README.md a rama de github Sprint - 2	Juan	Terminado	Repositorio rama sprint - 2
T3.5–T3.6	Pruebas de aceptación y persistencia/JSON dañado.	Marco	Pruebas añadidas al documento 08 de los .md	08_plan_de_pruebas.md
T3.7	PDF del informe final	Todos	PDF preparado y terminado	Docs/Informe_Final_Scrum_QPM_Clothes.pdf

Pruebas y estado de cierre

Actividades completadas: desarrollo e integración del código, persistencia de datos mediante archivos JSON, generación de reportes CSV, documentación de reglas de negocio, estructura del proyecto y guía de pruebas. También se finalizaron la documentación Scrum y la elaboración del informe PDF de cierre.
Pruebas y validación: se realizaron las verificaciones funcionales de la aplicación y se consolidó la evidencia de las pruebas. Se ajustaron las validaciones y las reglas de negocio para garantizar el comportamiento esperado de los pedidos y sus estados.
Documentación y entrega: se completaron las instrucciones de ejecución, el README final y los documentos del proyecto. Los archivos finales quedaron organizados en la rama de entrega, junto con las evidencias y los enlaces correspondientes al repositorio y al tablero de seguimiento.
Sprint Review: se realizó la revisión final del producto, comprobando el cumplimiento del objetivo del Sprint, las funcionalidades implementadas y los entregables definidos.
Retrospective: como resultado del proceso, se identificó la importancia de realizar una revisión cruzada del código, las pruebas y la documentación antes de cerrar las historias de usuario. Esta práctica se establece como una mejora para futuros sprints.

7. Conclusiones

QPM Clothes construye una aplicación de consola para centralizar y seguir pedidos de ropa, con módulos Python separados, persistencia JSON y reportes CSV. El producto implementa gran parte del Product Goal y muestra evolución en el repositorio mediante commits verificables.

El aprendizaje central del equipo es que la funcionalidad no se considera terminada solo porque exista código: debe integrarse, probarse, documentarse y vincularse a evidencia. La planificación Scrum aporta claridad de prioridades y responsabilidades, mientras que la revisión final detecta diferencias entre los documentos y el comportamiento ejecutable.

Evidencias principales

Repositorio: https://github.com/juanlsk29/QPM--CLOTHES